

Decision Log

Team: Zenith (HM26-7096) Sub-problem: Verification Date: 20 Sept 2026

Q1. What approach did we take, and what did we reject?

Our approach: every automated signal is advisory only — a human always makes the final call, and nothing auto-rejects.

The weighted proof score, the build check, the GitHub-mocking heuristic, and the AI candidate/interview summary all feed a submission's evidence file, but none of them can remove a candidate from view on their own. A failed build zeroes the score outright but the submission still routes to a human reviewer instead of disappearing; a flagged integrity event is logged with severity and left for a reviewer to weigh, never auto-blocked. Recruiters see the same evidence trail as everyone else — resume, GitHub, architecture notes, human reviews, institution verification and AI notes — side by side.

Alternative we considered and rejected: a fully automated pipeline where the proof score plus the AI interview transcript directly produced a pass/fail verdict with no human step in between.

It looked attractive at first because it removes the reviewer bottleneck entirely and gives recruiters an instant, ranked shortlist at any scale, with zero reviewer hours spent.

Q2. Why did we reject it? The trade-off

Dimension	Our approach	Rejected alternative
Accountability for a wrong call	A named human owns every reject/hire decision	No one is accountable when the model is wrong
Funnel speed	Slower — bounded by reviewer/recruiter bandwidth	Instant, no bottleneck at any volume
Recourse for false positives	A shaky build or heuristic flag gets a second look before it costs someone a shot	A candidate is silently dropped with no recourse
Build effort in 72h	Extra plumbing: review queues, flag routing, a “why this rank” endpoint	Simpler: one score, one threshold

Accountability decided it: recruiters told us in our own conversations that they would not act on a black-box score without being able to see why. We accept the cost — our funnel is slower and depends on reviewer and recruiter bandwidth, so on a busy day a pile of unreviewed submissions can sit waiting instead of resolving instantly.

Q3. What breaks at the scale of all of Mysuru?

What breaks first	Why (rough number)	How we'd fix it
Reviewer bandwidth	Doesn't scale linearly with submissions — a cohort of ~50 already needs several active reviewers; city-wide volume would need hundreds of expert-hours a week	ML-assisted reviewer triage/calibration, routing only borderline cases to a human
Simulated build runner	Single Flask process — more than a handful of concurrent submissions starts queuing or blocking	Isolated Docker/gVisor execution behind a job queue
Duplicate / near-duplicate work	GitHub-mocking check is a manual repo-age-vs-commits heuristic; policing thousands of repos by hand doesn't hold	Automated dedup/similarity pipeline across all submissions

The first change we'd make is ML-assisted triage that pre-sorts submissions so only genuinely borderline ones reach a human — keeping the human-in-the-loop principle for the decisions that matter, without making reviewer time the bottleneck for everyone else.